

Implementation-centric Classification of Business Rules from Documents

Preethu Rose Anish
TCS Research
Pune, India
preethu.rose@tcs.com

Abhishek Sainani
TCS Research
Pune, India
a.sainani@tcs.com

Abdul Ahmed
TCS Research
Pune, India
ahmed.abdul@tcs.com

Smita Ghaisas
TCS Research
Pune, India
smita.ghaisas@tcs.com

Abstract—In large multi-site multi-vendor projects, studying requirement documents to understand the problem domain and inferring possible solution to the posed problem is an important activity in Requirements Engineering. The process of reading User requirements Specification (URS) to create Software Requirement Specification (SRS) is a knowledge intensive activity that precedes several other important Software Engineering (SE) activities such as design and test plans. Automated Interpretation of the URS in terms of implementation-specific knowledge elements for software engineers' consumption has been reported in the past. The aim of such an interpretation is to reduce the effort associated with a manual extraction of knowledge elements and subsequently, their “translation” into primitives understood by those who must build the intended software. In this paper, we present a deep learning model for an implementation-centric classification of one such knowledge element, namely, business rules. We discuss an approach based on a Bidirectional Long Short Term Memory Network (BiLSTM) to capture the context information for each word, followed by an attention model to aggregate useful information from these words to get to the final classification. Our model adopts an end-to-end architecture that does not rely on any handcrafted features.

Index Terms—Business Rules, Classification, Deep Learning, BiLSTM, Implementation-centric.

I. INTRODUCTION

Software Engineering (SE) activities are usually globally distributed and carried out by multiple vendors. It is often likely that the User Requirements Specification (URS) is collected by one vendor and the corresponding Software Requirement Specification (SRS) may need to be created by another vendor. Under such circumstances, automating the interpretation of the URS in terms of implementation-specific knowledge elements would add a significant value by accelerating the process of creating SRS from URS. In this paper, we focus on an automated extraction and an implementation-centric classification of one such knowledge element, namely, business rules.

Business rules guide operations of businesses. They are an important source of information for software system implementation. From an information-system perspective, a business rule is a statement that defines or constrains some aspect of the business. It is intended to assert business structure, or to control or influence the behavior of the business [1]. Business rules are often found to exist in the form of both structured and unstructured text in requirements documents and other project documentation such as business agreements, charter documents,

feature descriptions, guidebooks and technical specifications. In our experience, rules are often found embedded in descriptions of business processes, use cases and workflows; rather than explicated independently [2]. Even when documents contain dedicated sections on business rules, they are likely to be found scattered across other locations making their comprehension a tedious task.

Several initiatives aim to address aspects such as organization, representation, and classification of rules. Though Semantics of Business Vocabulary and Rules (SBVR) has come up as a universally popular format for representation of business rules, there is no universal standard for classification of business rule [7, 8]. Our classification schema aligns with sub-types of the rules types defined by Ross [3]. Our implementation-centric classification schema is reported in [4]. In [4] we posited that each rule statement can be represented as a composition of atomic Rule intents. Rule intents are the intended constraints in the rule statement. We algorithmically identified groups of frequently co-occurring Rule intents and labelled them as Rule acts thereby classifying the rule statements along different Rule acts. However, this was a pattern-based approach wherein we relied on manual patterns creation based on combination of Parts of Speech (POS) tags, keywords and wild card characters. Having deployed this approach in practice, over a period of time, we realized that creation of such handcrafted features is an effort-intensive process. In addition, given the practical scenario, wherein large outsourcing vendors work on large projects in varying domains, a pattern-based approach is not likely to scale.

Traditional machine learning algorithms such as SVM or logistic regression are based on shallow models trained on sparse features and they suffer from the curse of dimensionality [5]. With the recent success of word embedding, neural network models have achieved superior results on various language-related tasks as compared to traditional machine learning models. Two Artificial Neural Network (ANN) architectures widely used in various Natural Language Processing (NLP) tasks are Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN). The former learns spatially based semantic representations by using locally connected layers, while the latter learns sequential-based features [22]. A limitation of traditional RNN is that it suffers from exploding or vanishing gradient features, which makes it hard to learn and tune the parameters of the earlier layers in the

network [24]. This is a particular concern in our case, given that unlike tweets/ product reviews, business rule sentences are often quite long. Long Short-Term Memory (LSTM) networks are known to overcome this limitation associated with traditional RNNs.

In this paper, we therefore investigate the effectiveness of using word embedding and LSTM model with attention mechanism for classification of business rules. In this method, first the words in the sentences in URS are converted to a vector form using a word-embedding technique [32]. Then the vector representation of each word in the sentence is provided as input to the LSTM model to learn the long distance contextual dependency among words. Next, the attention mechanism is applied for the model to identify and focus on the important parts of the sentence. Finally, based on the probability score provided by the output layer of the network, the sentence is assigned to an appropriate class. Based on the enhancements in the accuracies, we can say that deep learning methods can effectively identify business rules and perform business rule classification along the Rule acts. We use the pattern-based approach reported in [4] as our baseline for comparing the results of the LSTM based approach reported here.

The rest of the paper is organized as follows: Section II presents the classification schema for business rules, Section III details our method and presents the results, Section IV presents related work while section V concludes the paper.

II. CLASSIFICATION SCHEMA FOR BUSINESS RULES

In our earlier work [4], we introduced the concept of Rule acts to present an implementation-centric classification of business rules. In [4] we posited that the Rule acts are composed of frequently co-occurring Rule intents. Rule intents are atomic constraints embedded in a natural language rule statement. Each rule statement can therefore be represented as a composition of Rule intents. We then applied a clustering algorithm over the rule statements to detect the Rule intents that co-occur most frequently. The groups of frequently co-occurring intents were manually inspected to name the various “Rule acts”. Table 1 presents the Rule acts, their definition and sanitized examples. Rows colored *grey* are the Rule acts that are already reported in the our earlier work [4], and the others are the new Rule acts discovered as a result of deployment of our earlier work in real life projects in our organization.

TABLE I. RULE ACTS, DEFINITIONS AND EXAMPLES

Rule Act Name	Definition	Example
Deadline	Rules that restrict time duration, date.	Collection must be done by 25 th of every month.
Conditional Execution	Rules describing checks to be performed by user or system while executing process steps.	When a claim is intimated, current information held on the Exceed policy record should be passed through to <<system name>>.
Access Control	Rules that restrict stakeholder’s access or constraints activities performed by them.	The system only allows the Claim Officer to modify the Interim Claim payment amounts. The Claim Officer will not be allowed to alter anything else for the Claim in question.
User Interface	Rules related to user interface describing various screen elements, screen layout etc.	A drop down list of reason code for the cancellation should be provided and a reason code for the cancellation must be selected.
Data Validation	Rules that validate data processed by the system or used by stakeholders to perform some task.	Regular withdrawals will cease when the amount of withdrawal exceeds half the plan value, eg: Plan value = Rs. 2000 Withdrawal = Rs. 1050 not allowed. Only a withdrawal of Rs. 1000 would be allowed in this case.
Data Protection	Rule act that mandates the practices to be followed to protect sensitive data.	The policy number of the customer must be masked for unauthorized users.
Documentation Mandate	Rule act that mandates the information that needs to be provided in a document.	Documents with all relevant policy details must be retained on one system to allow for real time review and print.
Calculational	Rule act that describes the data relevant to the incumbent calculative processes.	Any return premium should be calculated on a pro rata basis as follows: Refund = (Next DD date less Cancellation Date)/30 X Monthly Premium; e.g., Cancellation Date is 27/8; next DD date is 7/9; Refund = 11/30 X MP.
Financial Transaction	Rule act that restricts the monetary transactions.	Any payments must be authorized or auto authorized prior to being passed to oracle.
User Responsibility	Rule act that mandates the user to perform some action.	Customer must advise if they have received notification that their job may be at risk.

Our experience in applying the approach reported in [4] on real-world SRS from varying domains including Insurance, Security and Healthcare, leads us to believe that Rule acts listed in Table 1 comprehensively represent implementation-specifics classification schema for business rules.

III. METHOD AND RESULTS

In this section, we present details of the dataset, the classifier architecture, experiments and results.

A. Dataset Preparation

Our dataset consists of 4043 sentences, of which 1676 are business rule and 2367, non-rules. These sentences are taken from 40 SRS documents from large projects in the Insurance domain. Out of these 40 documents, we already had annotations available for 30 documents (from our previous work [4]). Since deep learning models demand large datasets, we annotated 10 more SRS documents. This was done by one in-house subject matter expert. Deep learning experiments with dataset size similar to ours have been reported in the past and have produced good accuracies, for example, in works [22, 23]. We therefore posit that ours is a reasonable dataset for reliable deep learning experiments. The total number of instances of each Rule act is provided in Table II.

TABLE II. NUMBER OF INSTANCES OF RULE ACTS

Rule acts	Number of Instances
Deadline	65
Conditional Execution	560
Access Control	203
User Interface	449
Data Validation	299
Data Protection	45
Documentation Mandate	100
Calculational	152
Financial transaction	145
User Responsibility	295

B. Classifier Architecture

We propose a two-step classification method where the first classifier identifies business rules and the second classifier assigns one or more Rule acts to the identified business rule statement. Hereafter, we will refer to Rule acts as classes.

Our classifier architecture, for each of the two classifiers, consists of an embedding layer, followed by a BiLSTM layer, an Attention layer and an output layer (Fig. 1), hereafter collectively referred to as BLSTM-att.

Embedding Layer: We use Word2Vec model (Skipgram with negative sampling [33]) as the word embedding technique to initialize the embedding layer.

BiLSTM layer: We use Bidirectional LSTM (BiLSTM), instead of a simple LSTM model, to learn the contextual dependencies among words since it learns this information in both forward and reverse direction. This provides BiLSTM with the complete context for a given word.

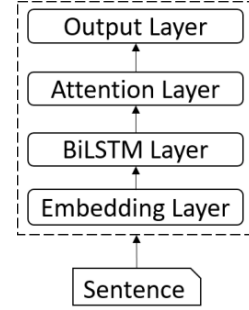


Fig. 1. BLSTM-att Classifier architecture

Attention layer: Output of the LSTM model is based on the output of its last node, which processes the last word in the sentence along with the context dependency information from previous nodes. Even though every node of LSTM remembers the information received from previous nodes, in case of long sentences not all information can be remembered effectively. Same is true in case of Bi-LSTM where we train the model in both the directions. Due to this, instead of relying on information from the last node, we want to obtain information at every node of the model. Using an attention mechanism, we do this by training a single-layer feed-forward neural network to learn the importance of each word in a given sentence within the context of the classes. Using this, we obtain the weights corresponding to each word based on the importance of the word in the sentence. Finally, instead of only obtaining the final score using the last layer of LSTM, we obtain the final score based on an aggregation over the weighted output of all the nodes.

Output layer: We use sigmoid function in the output layer for both identification of rule sentences (binary classification), and classification of a rule sentence into one or more Rule acts (multilabel classification). The sigmoid function is useful because it converts the final score assigned to the input sentence for a given class, to 1 if the score is 0.5 or higher, or 0 otherwise (Figure 2). Here 1 implies that the sentence belongs to the class and 0 implies otherwise. Thus, the first classification task uses a single sigmoid function, whereas the second classification task uses multiple sigmoid functions, one for each class.

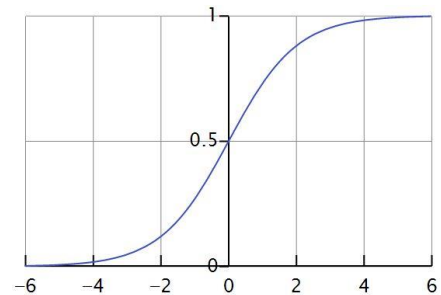


Fig. 2. Sigmoid function curve

Loss function: We use binary cross-entropy loss or Sigmoid cross-entropy loss as the loss function in our architecture, for both the classification tasks. Moreover, to address the class

imbalance problem of our dataset, we have used the weighted form of this loss function, as defined below:

$$WBCE = -w_1 * y * \log(p) - w_0 * (1-y) * \log(1-p),$$

where y is the class indicator (0 or 1), p is the predicted probability for a sentence to belong to class 1 and w_0 and w_1 are the weights for class 0 and class 1 respectively. Here, the value of w_0 and w_1 is the reciprocal of the number of sentences in their respective class.

C. Experiments and Results

In our experiments, we use a 5-fold cross validation technique to test the performance of our model. For each classification task, in each fold, we first initialize the embedding layer with a Word2Vec model trained on the training dataset. Then, we train BLSTM-att on the training data.

Experiment 1: In the first classification task, the training data is the set of both rule and non-rule sentences. The set of common hyper-parameters and their values that we have used in this experiment are mentioned in Table III.

TABLE III. PARAMETER VALUES FOR EXPERIMENT 1

Parameters	Value
Embedding Dimensions	50
Activation Function	Sigmoid
Bi-LSTM Dropout Rate	0.5
Bi-LSTM Recurrent Dropout Rate	0.5
Bi-LSTM Hidden Layers	32
Learning Rate	0.002
Epochs	50
Sentence length	36

Table IV shows the precision and recall values for BLSTM-att and baseline.

TABLE IV. RESULTS FOR EXPERIMENT 1

	Precision (in %)	Recall (in %)
Baseline	84	78
BLSTM-att	88.11	87.6

Experiment 2: In the second classification task, the training data is the set of rule sentences. The set of common hyper-parameters and their values that we have used in this experiment are mentioned in Table V.

TABLE V. PARAMETER VALUES FOR EXPERIMENT 2

Parameters	Value
Embedding Dimensions	50
Activation Function	Sigmoid
Bi-LSTM Dropout Rate	0.65
Bi-LSTM Recurrent Dropout Rate	0.65
Bi-LSTM Hidden Layers	64
Learning Rate	0.002

Epochs	300
Sentence length	40

In Table VI, we share the precision and recall value for each class obtained in this experiment.

TABLE VI. RESULTS OF EXPERIMENT 2

Class (Rule acts)	Precision (in %)	Recall (in %)
Deadline	91.65	74.99
Conditional Execution	84.09	77.89
Access Control	74.28	83.87
User Interface	92.47	80.95
Data Validation	55.36	49.02
Data Protection	99.9	49.9
Documentation Mandate	72.72	44.44
Calculational	71.42	80
Financial transaction	62.96	56.66
User Responsibility	70.69	74.54

Since in our previous work [4], we reported the precision for five classes, in Table VII, we present the comparison of results with only those classes between the baseline and BLSTM-att.

TABLE VII. COMPARISON WITH BASELINE

Class (Rule acts)	Baseline Precision (in %)	BLSTM-att Precision (in %)
Deadline	72.11	91.65
Conditional Execution	60.94	84.09
Access Control	52.54	74.28
User Interface	91.8	92.47
Data Validation	92.65	55.36

D. Discussion

The result of Experiment 1 (binary classification) clearly indicates that a BLSTM-att model performs better in identifying rule sentences from business documents, as compared to our previous work [4] that used linguistic pattern based method.

We observe a similar outcome in Experiment 2 (multilabel classification) where we obtain higher precision scores when using a BLSTM-att model than the precision scores reported in our previous work. The Rule act, *Data Validation*, however is an exception here. Upon analysis, we observed that *Data Validation* includes a broad spectrum of information that can itself be categorized into sub-categories of this Rule act. For instance, we observed that data validation related business rules need to be executed at various levels of any project – business level, user interface level, database level etc. The sentences corresponding to each of these types are articulated in a different manner in the URS. Therefore, such variations need to be addressed by segregating them at a lower granularity, which is what we plan to explore further in our future work.

Among the new Rule acts that we have introduced in this work, *Documentation Mandate* and *Data Protection* have low

recall, mainly due to low number of instances in the business documents that we used in our experiments.

We acknowledge that our comparison is not completely fair given that the dataset for the current experiment is larger than the one used in the baseline experiment. Nevertheless, we find our approach to be better given the notable increase in the scores coupled with the fact that it eliminates the task of manual pattern creation thereby promoting scalability.

IV. RELATED WORK

Business Rules are an integral part of the requirements analysis process. A large body of knowledge has emerged as a result of their significance to software engineering. Most of the work in this area attempts to better understand the business rules by focusing on (1) definition, (2) extraction, (3) classification, and (4) management. Of these, only extraction and classification are relevant to be discussed in the context of the technique employed in our work.

Extraction: A considerable amount of work in the area of extracting business rules from source code, databases, event logs and legacy systems has been reported in the literature. Since, our focus is on requirements specification phase, we restrict our literature review to only work done on extracting business rules from natural language text. In [6], the authors proposed an approach for automatic extraction of business rules from unrestricted text in the Banking industry. Their approach relied on a rule detection component, which was built on a set of linguistically motivated rules or patterns inspired in SBVR and RuleSpeak languages. Such pattern-based approaches are effort intensive. Further, from our industrial experience, we have observed that business rules are never written following any specific standards. Therefore, approaches based on linguistic rules and patterns are not likely to scale. The work reported in [9] on extraction of business rules from business processes specifications written in natural language also relies on a set of business rule heuristics, linguistic patterns and keywords and therefore has the same drawbacks as argued for [6]. Breaux and Antón [10] proposed a method to mine rule semantics to understand legislative compliance. The method was applied on privacy regulations derived from the Health Insurance Portability and Accountability Act (HIPAA). They identified patterns for privacy rules in HIPAA. They manually extracted patterns for temporal constraints (that equate to Deadline in our classification), pre-condition (that equates to Condition in our classification), Cardinality (in our classification schema, this is divided into two sub-classes namely Data validation and Calculational), Arithmetic operators (compares to Calculational class in our schema), Comparison relations (a superset of class Conditional Execution in our schema) and Ordinality. However, the method is restricted to manual identification of HIPAA constraint types using patterns. Xiao et al. [11] proposed a technique called Text2Policy for automated extraction of access control policies from natural-language software documents. They designed an ACP model and used shallow parsing and domain dictionary to extract ACP model instances and represent them in an eXtensible Access Control Markup Language (XACML) specification. While their technique produces good results, it works only for

extracting access control policies, which is one of many Rule acts that we have in our classification schema. Moreover, it relies heavily on the ACP model and the domain dictionary, and it would take time and effort to extend this technique to other Rule acts. In [12] the authors proposed a Requirements-based Access Control Analysis and Policy Specification (ReCAPS) method for deriving access control policies from various sources, including software requirements specifications (SRS), software designs, and high-level security/privacy policies. It is a manual analysis method supported by heuristics. In [13], the authors proposed a linguistically oriented, rule-based approach to extract deontic rules from regulations. Alshugran and Dichter [14] proposed a method to analyze, extract, and model the access control rules in HIPAA. However, their method is manual. In [15], the authors extracted business rules from legal text by combining the linguistic information provided by WordNet together with a syntax-based extraction of rules, and a logic-based extraction of dependencies between chunks of such texts. However, their work is heavily dependent on lightweight ontologies that either need to be created or extended for a given use-case. Slankas and Williams [16] proposed a machine-learning based approach to parse existing, unaltered natural language documents, such as requirement or technical specifications to extract the relevant subjects, actions, and resources for an access control policy. For extracting the access control rules, they use a combination of K-NN classifier, naïve Bayes, and an SVM classifier. However, unlike as in our case, they focused only on Access control policies and not the entire implementation-centric classification spectrum.

As is evident from the above discussion, a considerable amount of work exists in the area of extraction of business rules from natural language text. However, all of them rely on pattern-based approaches or attack a very narrow slice of the problem. To the best of our knowledge, there is no reported work on utilizing state-of-art deep learning technique for business rule extraction from natural language text.

Classification: Since we have already defended the novelty of our classification schema in our previous work [4], we do not include here the various business rule classification viewpoints that exist in the literature. Since the contribution of this paper is the state-of-art technique that we used for the implementation centric classification of business rules, we searched the literature to find automation efforts in classifying business rules. We found related work on automated classification of requirements artifacts such as non-functional requirements [17, 18, 19, 20], architecturally significant functional requirements [20], classification of requirements as ‘requirement’ or ‘information’ using CNN [30], temporal and non-temporal requirement, functional or the non-functional categories [31]. To the best of our knowledge, there is no reported work on automating the classification of business rules from project documents.

V. CONCLUSION

The Requirements Engineering phase of software development involves an effort intensive activity of reading URS to create SRS. Interpreting the URS in terms of implementation-specific knowledge elements aids in reducing the effort associ-

ated with a manual extraction of knowledge elements and subsequently, their “translation” into primitives understood by those who must build the intended software. This paper has presented a deep learning architecture for extracting business rules from URS and classifying them into different implementation-centric classes. We obtained a precision and recall of 88.1 % and 87.6% respectively for the binary classification task which shows a decent improvement over our baseline approach where we obtained a precision of 84% and a recall of 78%. Similarly, for the multilabel classification as well we see an improvement in scores except for *Data Validation*, which we plan to further explore in future.

The results indicate that deep learning techniques can produce equivalent or better results for the rule classification task and eliminate the need for manual creation of patterns thereby saving the effort involved in learning and writing patterns. Further, since deep learning techniques such as LSTM learn the contextual dependencies among words in a sentence, they are likely better scalable.

Further performance improvements may be possible by considering more advanced LSTM variants such as deeper self-attention mechanisms [25], stacking BILSTMs [26] or pre-training the BILSTMs [27] with auxiliary tasks. We would also like to explore pre-trained language models such as ULMFiT [28] and BERT [29] in the future.

REFERENCES

- [1] Business Rules Group (2001). WWW: <http://www.businessrulesgroup.org/brgdefn.htm>. Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955.
- [2] Maalej, W., & Ghaisas, S. (2014). Capturing and sharing domain knowledge with business rules lessons learned from a global software vendor. In *Requirements engineering conference (RE), 2014 IEEE 22nd International*. IEEE (pp. 364–373).
- [3] R. G. Ross, *The Business Rule Book: Classifying, Defining and Modeling Rules*, Business Rule Solutions Inc; 2nd edition, 1997
- [4] S. Ghaisas, M. Motwani and P. R. Anish, "Detecting system use cases and validations from documents," *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Silicon Valley, CA, 2013, pp. 568-573
- [5] Young, T., Hazarika, D., Poria, S. and Cambria, E., 2018. Recent trends in deep learning based natural language processing. *ieee Computational intelligence magazine*, 13(3), pp.55-75
- [6] Martínez-Fernández, J.L., González, J.C., Villena, J. and Martínez, P., 2008, June. A preliminary approach to the automatic extraction of business rules from unrestricted text in the banking industry. In *International Conference on Application of Natural Language to Information Systems* (pp. 299-310). Springer, Berlin, Heidelberg.
- [7] Barbara von Halle, (2002), what is a Business Rules Approach? Published: January 1, 2002
- [8] Barbara von Halle, (2002), *Business Rules Applied—Business Better Systems, Using the Business Rules Approach*, Published by John Wiley & Sons, Inc., New York
- [9] Tobon, H.G. and Franco, A., 2010. *Business rules extraction from business process specifications written in natural language*. *Business Rules Journal*, 11(7).
- [10] T. Breaux, and A.I. Antón, Mining rule semantics to understand legislative compliance, *Proceedings of ACM workshop on Privacy in the electronic society*, November 2005, Alexandria, VA, USA
- [11] X. Xiao, A. Paradkar, S. Thummalapenta, and T. Xie. “Automated extraction of security policies from natural-language software documents”. *Proceedings of 20th International Symposium on the Foundations of Software Engineering (FSE)*, pp. 12:1–12:11, 2012
- [12] Q. He and A. I. Antón. Requirements-based access Control Analysis and Policy Specification (ReCAPS). *Inf. Softw. Technol.*, 51(6):993–1009, 2009
- [13] Wyner, A.Z. and Peters, W., 2011, December. On Rule Extraction from Regulations. In *JURIX* (Vol. 11, pp. 113-122).
- [14] Alshugran, T. and Dichter, J., 2014, May. Extracting and modeling the privacy requirements from HIPAA for healthcare applications. In *IEEE Long Island Systems, Applications and Technology (LISAT) Conference 2014* (pp. 1-5). IEEE.
- [15] M. Dragoni, S. Villata, W. Rizzi, G. Governatori. Combining NLP Approaches for Rule Extraction from Legal Documents. 1st Workshop on Mining and Reasoning with Legal texts (MIREL 2016), Dec 2016, Sophia Antipolis, France.
- [16] Slankas, J. and Williams, L., 2013, September. Access control policy extraction from unconstrained natural language text. In *2013 International Conference on Social Computing* (pp. 435-440). IEEE.
- [17] J. C.-Huang, R. Settini, X. Zou, P.Solc, “Automated classification of non-functional requirements”, *Proceedings of IEEE International Requirements Engineering conference*, vol.12, pp.103-120, May 2007
- [18] Casamayor, A., Godoy, D. and Campo, M., 2010. Identification of non-functional requirements in textual specifications: A semi-supervised learning approach. *Information and Software Technology*, 52(4), pp.436-445.
- [19] Slankas, J. and Williams, L., 2013, May. Automated extraction of non-functional requirements in available documentation. In *2013 1st International Workshop on Natural Language Analysis in Software Engineering (NaturaLiSE)* (pp. 9-16). IEEE.
- [20] Mahmoud, A. and Williams, G., 2016. Detecting, classifying, and tracing non-functional software requirements. *Requirements Engineering*, 21(3), pp.357-381.
- [21] Anish, P.R., Balasubramaniam, B., Sainani, A., Cleland-Huang, J., Daneva, M., Wieringa, R.J. and Ghaisas, S., 2016, May. Probing for requirements knowledge to stimulate architectural thinking. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)* (pp. 843-854). IEEE.
- [22] Neill, J.O., Buitelaar, P., Robin, C. and Brien, L.O., 2017, June. Classifying Sentential Modality in Legal Language: A Use case in Financial Regulations, Acts and Directives. In *Proceedings of the 16th edition of the International Conference on Artificial Intelligence and Law* (pp. 159-168). ACM
- [23] Marasović, A. and Frank, A., 2016. Multilingual modal sense classification using a convolutional neural network. *arXiv preprint arXiv:1608.05243*.
- [24] Young, T., Hazarika, D., Poria, S. and Cambria, E., 2018. Recent trends in deep learning based natural language processing. *ieee Computational intelligence magazine*, 13(3), pp.55-75.
- [25] J. Pavlopoulos, P. Malakasiotis, and I. Androutsopoulos. 2017. Deeper attention to abusive user content moderation. In

- Proceedings of the Conference on Empirical Methods in Natural Language Processing, pages 1125–1135, Copenhagen, Denmark
- [26] O. Irsoy and C. Cardie. 2014. Deep recursive neural networks for compositionality in language. In Proceedings of the 27th International Conference on Neural Information Processing Systems, pages 2096–2104, Montreal, Canada
 - [27] P.Ramachandran, P.r.J.Liu, and Q.V.Le. 2017. Unsupervised pretraining for sequence-to-sequence learning. In Proceedings of the Conference on Empirical Methods in Natural Language Processing, pages 383–391, Copenhagen, Denmark.
 - [28] Howard J, Ruder S. *Universal Language Model Fine-tuning for Text Classification*. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) 2018 (Vol. 1, pp. 328-339).
 - [29] Devlin, J., Chang, M.W., Lee, K. and Toutanova, K., 2018. *Bert: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv preprint arXiv:1810.04805.
 - [30] Winkler, J. and Vogelsang, A., 2016, September. Automatic classification of requirements based on convolutional neural networks. In *2016 IEEE 24th International Requirements Engineering Conference Workshops (REW)* (pp. 39-45). IEEE.
 - [31] J. H. Hayes, W. Li, and M. Rahimi, “Weka meets TraceLab: Toward Convenient Classification: Machine Learning for Requirements Engineering Problems: A Position Paper,” in 1st IEEE International Workshop on Artificial Intelligence for Requirements Engineering, ser. AIRE, 2014, pp. 9–12.
 - [32] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.
 - [33] Goldberg, Y., & Levy, O. (2014). word2vec Explained: deriving Mikolov et al.'s negative-sampling word-embedding method. arXiv preprint arXiv:1402.3722